PUBSUB THEORY Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

1. What are the roles of publishers, topics, and subscribers in Pub/Sub?

In Pub/Sub, everything revolves around decoupling producers and consumers.

- **Publisher**: This is the application or service that produces messages. For example, an IoT sensor app might publish temperature readings, or an e-commerce app might publish order events. The publisher just sends messages, it doesn't care who consumes them.
- **Topic**: This acts like a logical channel or pipeline where messages are collected. Publishers send messages into a topic. Topics allow multiple subscribers to receive the same data stream without the publisher having to send messages to each one individually.
- **Subscriber**: These are consumers that subscribe to the topic. A subscriber could be a Dataflow pipeline, a Cloud Function, or even a custom application. Subscribers receive messages and process them as needed.

The nice part is that Pub/Sub creates loose coupling. Publishers and subscribers don't need to know each other. This makes the system more scalable and flexible — publishers can send millions of events per second without worrying about downstream load.

2. What is the difference between push and pull subscriptions in Pub/Sub?

Subscriptions are how subscribers connect to a topic to receive messages, but there are two modes:

- **Pull subscription**: In this mode, the subscriber explicitly calls Pub/Sub to fetch messages. It's like polling. The subscriber has control over how many messages to pull and when to pull. This is useful if the subscriber system can't always handle a continuous flow or if you need to control backpressure. For example, a batch processing system can pull messages in controlled batches.
- **Push subscription**: In this mode, Pub/Sub automatically pushes messages to a configured endpoint (usually an HTTPS endpoint like a Cloud Function, Cloud Run service, or API endpoint). This is event-driven and has lower latency because messages are delivered immediately as they arrive. The trade-off is that the subscriber endpoint must always be available to receive requests.

So pull = subscriber-driven (controlled pace), push = Pub/Sub-driven (fast and automatic). In real-world systems, pull is common for heavy pipelines like Dataflow, while push is common for lightweight event-driven tasks like triggering a function.

3. How does message acknowledgment work in Pub/Sub?

Acknowledgment is the key to ensuring reliable delivery.

When Pub/Sub delivers a message to a subscriber (whether push or pull), it waits for the subscriber to confirm that the message was processed successfully by sending back an "ack." Only then does Pub/Sub mark the message as delivered and remove it from the subscription's queue.

If the subscriber does not ack within the ack deadline (default is 10 seconds, configurable up to 10 minutes), Pub/Sub assumes the message wasn't processed and will redeliver it. This guarantees no messages are lost even if the subscriber fails or crashes.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Subscribers can also extend the ack deadline if they need more time, for example, if the message is large or processing is complex. If they ack early, the message is immediately cleared.

This ack mechanism ensures durability, but it also means subscribers must be idempotent because the same message might get delivered more than once if ack timing is missed.

4. What is the delivery guarantee model of Pub/Sub?

Pub/Sub is designed for **at-least-once delivery**. That means every message published to a topic will be delivered to subscribers at least one time, but sometimes a message may be delivered more than once. Duplicates can happen due to network retries, subscriber crashes, or missed ack deadlines.

Because of this, subscribers must be designed to handle duplicates safely. This usually means making the processing idempotent — for example, checking if a record with the same ID already exists before inserting into a database.

Some integrations, like Pub/Sub with Dataflow, can achieve effectively-once processing, but the base guarantee is at-least-once. There is no “at-most-once” model, because reliability and durability are more important than missing messages.

5. How are ordering keys used to preserve message sequence in Pub/Sub?

By default, Pub/Sub does not guarantee message ordering because it prioritizes scalability and parallelism. Messages may arrive out of order across subscribers.

If strict ordering is important, I can use ordering keys. When publishing, I attach an ordering key (for example, customer_id, order_id, or device_id) to the message. Pub/Sub then guarantees that all messages with the same ordering key are delivered in the exact sequence they were published.

For example, if I publish order events for customer123 with an ordering key = customer123, all those events will be delivered to the subscriber in the correct order, even if other customers’ events are being processed in parallel.

The limitation is that ordering is guaranteed only per key, not across all messages in the topic. This allows Pub/Sub to maintain order where needed while still scaling horizontally.

6. How can Pub/Sub be used to collect real-time IoT sensor data?

In IoT systems, thousands or millions of devices constantly generate small messages like temperature readings, GPS locations, or machine metrics. These devices or edge gateways can publish messages directly to a Pub/Sub topic. Pub/Sub then acts as a scalable buffer that can handle high throughput and bursts in traffic.

Subscribers such as Dataflow pipelines or Cloud Functions consume these messages in real time. Dataflow can aggregate or transform the sensor data and write it to GCS or BigQuery, while Cloud Functions can trigger alerts or downstream processes.

The advantage is that Pub/Sub decouples the producers (sensors) from the consumers (processing systems), making the pipeline more resilient and scalable for IoT workloads.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. Why is Pub/Sub often the entry point for streaming pipelines?

Pub/Sub is designed to ingest streams of events at scale with low latency. It can handle millions of messages per second reliably. For streaming pipelines, you need a system that can buffer events, guarantee delivery, and scale automatically. Pub/Sub provides exactly that.

It also integrates natively with Dataflow, Cloud Functions, Cloud Run, and external subscribers. This makes it a natural entry point because publishers just write to Pub/Sub, and multiple consumers can subscribe and process the same stream for different use cases (analytics, alerts, storage, ML).

So Pub/Sub sits at the front of streaming architectures, ensuring durability, scalability, and fan-out to multiple downstream pipelines.

8. What is the difference between batch ingestion (GCS/BigQuery) and streaming ingestion (Pub/Sub)?

Batch ingestion means collecting data in bulk at scheduled intervals. For example, uploading a daily CSV to GCS and loading it into BigQuery. It's simpler and cheaper but data freshness is limited because users only see new data once the batch job completes.

Streaming ingestion means data flows continuously, event by event, into Pub/Sub. Subscribers process it in real time, and data can land in BigQuery or GCS within seconds. This provides near real-time insights but is more complex and often more expensive since the pipeline runs continuously.

In short: batch = delayed but simpler and cost-effective, streaming = real-time but more complex to operate.

9. What is a hybrid ingestion strategy, and when would you use it?

A hybrid ingestion strategy combines both batch and streaming. It's useful when some data must be available in real time, while other data can be processed later in bulk.

For example, in an e-commerce system, real-time events like payments or fraud detection go through Pub/Sub streaming to BigQuery within seconds. At the same time, nightly product catalog updates or historical backfills are ingested as batch files into GCS and then loaded into BigQuery.

Hybrid strategies balance cost and complexity by giving real-time insights where needed and using cheaper batch loads where freshness is less critical.

10. How can Pub/Sub integrate with external data sources like Kafka or APIs?

Pub/Sub can connect with external systems in several ways. For Kafka, Google provides Pub/Sub Kafka connectors through Dataflow templates or third-party tools like Confluent, which can sync topics between Kafka and Pub/Sub. This is common when migrating workloads from on-prem to GCP or building hybrid systems.

For APIs, Pub/Sub can receive data through Cloud Functions, Cloud Run, or custom publishers. For example, an external system can call a REST API that publishes data into Pub/Sub. Similarly, subscribers can expose endpoints that receive push messages from Pub/Sub.

This integration allows Pub/Sub to act as a universal messaging backbone that connects cloud-native pipelines with legacy or external systems.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. How can Pub/Sub be used as a source for Dataflow pipelines?

Pub/Sub is one of the most common sources for Dataflow pipelines. I can create a streaming Dataflow job that subscribes to a Pub/Sub topic or subscription. As messages arrive in real time, Dataflow consumes them and applies transformations like filtering, parsing, aggregations, or enrichment.

For example, if IoT devices publish temperature readings into a Pub/Sub topic, a Dataflow pipeline can subscribe to it, clean the data, and write results into GCS or BigQuery. Pub/Sub provides the durability and buffering, while Dataflow does the scalable stream processing.

This combination is powerful because it delivers real-time ETL capabilities at scale.

12. How can Dataflow write transformed messages back into Pub/Sub topics?

Dataflow not only reads from Pub/Sub, it can also publish results to another Pub/Sub topic. After applying transformations, the pipeline can use the Pub/SubIO write connector to publish new messages.

This is useful for building event-driven architectures. For example, raw clickstream events can be read from one topic, processed in Dataflow to enrich with session details, and then written back into a new Pub/Sub topic for downstream consumers like fraud detection or personalization services.

This pattern allows chaining multiple pipelines together while keeping everything loosely coupled.

13. What windowing strategies are available in Dataflow for Pub/Sub messages?

Since Pub/Sub streams are unbounded, Dataflow uses windowing to group messages into logical chunks. The main strategies are:

- Fixed windows: Groups messages into fixed time intervals, like every 5 minutes. Useful for regular reporting.
- Sliding windows: Overlapping time windows, like a 5-minute window sliding every 1 minute. Useful for rolling averages.
- Session windows: Group messages by periods of activity separated by gaps of inactivity. Useful for user session analysis.
- Global window: One single window that includes all data (rare for unbounded streams but sometimes used with triggers).

Choosing the right window depends on the analytics or aggregation needed.

14. What role do triggers play when processing Pub/Sub messages in Dataflow?

Triggers decide when the results of a window are emitted. By default, results are emitted when a window closes, but triggers allow more flexibility.

For example, I can set an early trigger to emit partial results before the window closes, or late triggers to handle stragglers (messages that arrive after the window end). I can also use repeated triggers so results update continuously as more messages arrive.

This is important for real-time use cases, where waiting until the end of a window is too late. Triggers let me balance timeliness and completeness of results.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

15. How can Dataflow handle late or out-of-order messages from Pub/Sub?

Late or out-of-order messages are common in streaming because of network delays or retries. Dataflow handles this using watermarks and allowed lateness.

The watermark is an estimate of event-time progress. If messages arrive after the watermark has passed, they are considered late. I can configure allowed lateness to still accept these late messages for a certain period, like 10 minutes after the window closed.

If lateness is allowed, Dataflow will update previous results to include the late data. If not, those messages can be sent to a side output or dead-letter path for review.

This ensures that pipelines remain robust even when messages don't arrive in perfect order.

16. How can Pub/Sub messages be streamed into BigQuery?

There are two main ways. The first is using Dataflow: a Dataflow pipeline subscribes to a Pub/Sub topic, transforms the messages, and writes them into BigQuery tables using the BigQueryIO connector. This approach gives flexibility to apply parsing, validation, or enrichment before loading.

The second way is to use the BigQuery Storage Write API. Pub/Sub messages can be written directly into BigQuery through this API with very low latency. This avoids the overhead of building a custom pipeline, but it gives less control over transformations.

In practice, I choose Dataflow when transformations are required, and Storage Write API when I need minimal latency and can directly map messages into BigQuery.

17. What is the Storage Write API, and how does it compare to Dataflow ingestion?

The Storage Write API is BigQuery's streaming ingestion API. It lets me write rows into BigQuery with high throughput and lower cost compared to the older streaming insert API. It also supports exactly-once guarantees when used with Dataflow templates.

Compared to Dataflow ingestion, the Storage Write API is simpler and faster if my goal is just to land raw Pub/Sub messages into BigQuery. But Dataflow gives me more flexibility to validate, enrich, or filter data before loading.

So I think of it this way: Storage Write API = direct, low-latency load. Dataflow ingestion = flexible ETL pipeline with more control.

18. How can JSON or Avro Pub/Sub messages be mapped to BigQuery schemas?

If the messages are JSON, I need to parse the JSON fields and map them to the corresponding BigQuery columns. This can be done in Dataflow using a custom schema mapping or by configuring the pipeline to auto-map JSON keys.

For Avro messages, the mapping is easier because Avro already carries schema information. BigQuery can natively understand Avro schema and map it directly to the table's schema. This reduces errors and is more efficient.

In both cases, I need to make sure the field names, data types, and nested structures in the message match the BigQuery table schema, otherwise ingestion will fail.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

19. What is the difference between micro-batch and streaming ingestion into BigQuery?

- Streaming ingestion means rows are written into BigQuery almost immediately as they arrive, using the streaming API or Storage Write API. This gives very low latency (seconds) but is slightly more expensive per row.
- Micro-batch ingestion means buffering messages for a short interval (like 1–5 minutes) and then writing them into BigQuery in bulk load jobs. This reduces cost and is more efficient for large volumes, but introduces a small delay.

So the difference is latency versus cost. Streaming is best when real-time insights are needed, while micro-batch is a good balance for pipelines that can tolerate a slight delay.

20. How can schema evolution be handled in a Pub/Sub → BigQuery pipeline?

Schema evolution means the message structure changes over time, like adding new fields. To handle this, I usually design the pipeline with flexibility:

- Use nullable fields in BigQuery so new fields can be added without breaking existing queries.
- For JSON messages, parse them dynamically and map only known fields to BigQuery, while sending unknown fields to an audit log.
- For Avro, rely on its built-in schema evolution support — new optional fields can be added without breaking old data.
- Enable schema update options in BigQuery load jobs (ALLOW_FIELD_ADDITION) if batch loading is used.

The key is to keep raw messages in GCS or a raw BigQuery table, so even if the schema changes, I can replay and reprocess them later with the new schema.

21. How can Pub/Sub data be written into GCS for archiving?

The most common approach is to use a Dataflow streaming pipeline. The pipeline subscribes to a Pub/Sub topic, reads incoming messages, and writes them into GCS. To avoid writing one file per message, the pipeline uses windowing (like 5 or 15 minutes) to group messages into larger files before saving them.

This allows me to store a durable archive of all events in GCS for compliance, replay, or batch analytics later. Alternatively, if transformations are not needed, there are ready-made Dataflow templates (Pub/Sub to GCS) that make this easy to set up.

22. Why is Avro or Parquet preferred when storing Pub/Sub messages in GCS?

Pub/Sub messages are often small and frequent. If I store them as CSV or JSON, the files become larger, slower to process, and don't preserve schema well.

Avro and Parquet are better choices because they are binary formats that support compression and schema. Avro is row-based, so it's good for streaming ingestion because it writes quickly and preserves schema evolution. Parquet is columnar, which makes queries faster later in BigQuery or Spark, because only the needed columns are read.

Using these formats reduces storage cost, improves downstream performance, and keeps schema intact across pipelines.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

23. How can you design partitioning strategies for Pub/Sub → GCS pipelines?

The most common partitioning is time-based, since streaming data is continuous. I usually organize data into folders like year=2025/month=09/day=11/hour=14/. This makes it easy for downstream systems like BigQuery or Spark to filter only the relevant partitions.

I can also add attribute-based partitions if Pub/Sub messages carry metadata, like region, device type, or customer ID. For example: region=us/year=2025/month=09/day=11/.

The goal is to design partitions that match common query filters, so queries read only what's needed instead of scanning everything.

24. Why is the small files problem an issue for Pub/Sub → GCS pipelines?

Pub/Sub sends many small messages, and if each message is written directly into GCS, the result is thousands or millions of tiny files. This causes overhead in Spark, Dataflow, or BigQuery because every file has to be opened and read separately.

Small files also increase storage operations and cost. For example, processing 10,000 small files is slower and more expensive than processing 10 large files of the same size.

That's why Pub/Sub → GCS pipelines use windowing and sharding to batch messages into larger files before writing.

25. How can event-driven ingestion be triggered when files land in GCS from Pub/Sub?

Once Pub/Sub messages are written to GCS (via Dataflow or another pipeline), I can configure **GCS event notifications**. When a new file lands in a bucket, GCS publishes an event to a Pub/Sub topic. That topic can then trigger a Cloud Function, Cloud Run service, or Composer DAG to process the file further.

For example, raw Pub/Sub logs land in GCS every 5 minutes. As soon as the file is written, a Cloud Function triggers a BigQuery load job or a Dataflow batch pipeline for transformations.

This event-driven design makes the pipeline fully automated, with no need for manual scheduling.

26. How can Spark Streaming on Dataproc consume Pub/Sub messages?

Spark Streaming on Dataproc can consume Pub/Sub messages using the Pub/Sub connector provided by Google. With this connector, Spark treats Pub/Sub as a streaming source just like Kafka. The job subscribes to a Pub/Sub subscription and continuously pulls messages into Spark micro-batches.

Once the messages are inside Spark, I can parse them, apply schema, and run transformations like filtering, aggregations, or joins. The output can then be written to sinks like GCS, BigQuery, or even back into Pub/Sub.

There are two patterns here. The first is direct consumption, where Spark pulls from Pub/Sub using the connector. The second is indirect, where Pub/Sub feeds into GCS via Dataflow or a template, and Spark on Dataproc processes those files in streaming mode.

Direct consumption is better when I want low latency, but it requires careful handling of acknowledgments and checkpointing. Indirect via GCS is easier operationally, because Spark can just read windowed files, but latency is slightly higher.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. When would you combine Pub/Sub streaming with Dataproc batch jobs?

I combine Pub/Sub with Dataproc batch jobs when I need both real-time capture and large-scale batch analytics. Pub/Sub gives me the ability to capture and buffer events in real time, while Dataproc is more efficient for heavy processing like joins across huge datasets, aggregations, or machine learning preparation.

A typical example is clickstream analysis. I can capture user clicks in real time using Pub/Sub, and Dataflow or a small streaming job stores those events in GCS. Later, a Dataproc batch job runs nightly to join clickstream events with historical data, enrich them with reference datasets, and build training datasets for ML models.

This approach balances cost and efficiency. Real-time Pub/Sub pipelines handle immediate needs like alerts or dashboards, and Dataproc batch jobs handle deeper, large-scale analysis that doesn't need second-level latency.

28. How can Pub/Sub events be enriched with reference data in Dataproc?

Pub/Sub events usually carry minimal information, like IDs or codes. To make them useful, I enrich them with additional attributes from reference datasets. On Dataproc, I can do this inside Spark jobs.

If the reference dataset is small, I can broadcast it in Spark so that every worker has a copy. This way, when events stream in, Spark can join them with the broadcast data quickly. For example, if Pub/Sub sends product_id, I can enrich it with product name and category.

If the reference dataset is large, I can keep it in GCS or BigQuery and join in Spark. In streaming pipelines, I might also use external stores like Bigtable or Redis to serve reference lookups at scale.

To keep data fresh, I reload or refresh the reference dataset periodically into Spark. This ensures that even if metadata changes (like price or region mapping), the enrichment logic stays up to date.

29. What are the latency vs throughput trade-offs between Spark Streaming and Dataflow for Pub/Sub?

Dataflow is designed for continuous streaming. It processes events as soon as they arrive, giving very low latency, usually seconds. It has strong support for event-time processing, watermarks, triggers, and late data handling. The trade-off is that running continuous pipelines can be more expensive, since they are always on.

Spark Streaming works in micro-batches. The latency depends on the batch interval I set — for example, if the batch interval is 1 minute, then data will be available with about 1-minute delay. The benefit is that Spark is very good at high-throughput processing, heavy shuffles, and complex transformations, so it's often chosen when latency is less critical but processing needs are complex.

So the trade-off is: Dataflow = lower latency, simpler scaling, but higher continuous cost. Spark Streaming = slightly higher latency, but excellent throughput and flexibility for advanced analytics.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

30. Why are ephemeral clusters often used for Pub/Sub batch processing with Dataproc?

Ephemeral clusters are temporary clusters created only when a job needs to run, and deleted immediately afterward. They are very useful for Pub/Sub batch processing because data usually lands in GCS from Pub/Sub in small windows, like every 15 minutes or every hour.

Instead of keeping a Dataproc cluster running all day, I can create a cluster only when a batch of data is ready, process it with Spark, write results to BigQuery or GCS, and then shut down the cluster. This way I only pay for the processing time, not idle time.

Ephemeral clusters also ensure reproducibility — each run starts with a fresh environment, so there's no dependency drift. They can be customized per workload, meaning I can spin up large clusters for heavy jobs and smaller ones for lighter jobs.

This design pattern saves costs, improves flexibility, and ensures that Pub/Sub batch pipelines remain efficient even as data volume grows.

31. How can Composer be triggered by Pub/Sub events?

Composer itself doesn't subscribe directly to Pub/Sub. The usual pattern is that I configure Pub/Sub to trigger a Cloud Function or a Cloud Run service whenever a new message arrives. That Cloud Function then calls the Airflow REST API inside Composer to trigger the DAG.

This way, Composer DAGs don't have to poll for events — they become event-driven. For example, if Pub/Sub receives an event about a file landing in GCS, that event can trigger a Cloud Function, which then triggers the Composer DAG to start downstream processing.

This pattern connects event-driven ingestion with orchestration workflows.

32. How would you orchestrate a pipeline that starts with Pub/Sub ingestion?

I would design it like this:

1. Pub/Sub topic collects messages from producers (e.g., IoT sensors or application events).
2. A Dataflow pipeline subscribes to that topic and processes the events in real time (transformations, filtering, windowing).
3. Dataflow writes the processed output into sinks like GCS (for archiving) or BigQuery (for analytics).
4. Composer orchestrates the pipeline — for example, it can trigger the Dataflow job, monitor its execution, and perform post-processing tasks like validation or archiving.

Composer doesn't replace Pub/Sub or Dataflow, but it acts as the "controller" that ensures the pipeline starts correctly, runs in the right order, and integrates with other workflows like downstream reporting jobs.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

33. How can Composer monitor and retry Pub/Sub pipelines?

In Airflow (which powers Composer), I can set tasks that monitor the health of Pub/Sub and Dataflow jobs. For example:

- Use a Pub/Sub sensor to check if messages are flowing into a subscription.
- Use Dataflow operators in Composer to start jobs and track their status.
- Configure retries with exponential backoff in the DAG so if a task fails, Composer automatically retries it.

If a Pub/Sub → Dataflow job fails due to a temporary issue, Composer can restart the Dataflow pipeline or rerun the job. I can also integrate with Cloud Logging and Cloud Monitoring to set up alerts when message backlog grows too large, and Composer can trigger cleanup or recovery tasks automatically.

34. What is the role of DLQ routing in Pub/Sub orchestration with Composer?

DLQ (Dead Letter Queue) is where Pub/Sub routes messages that cannot be successfully delivered after multiple attempts. This prevents bad messages from blocking the pipeline.

In a Pub/Sub → Composer architecture, DLQ is very important because Composer can orchestrate workflows to process those DLQ messages. For example, a Composer DAG might periodically pick up messages from a DLQ subscription, validate them, and either reprocess them or send alerts to the operations team.

This ensures no data is silently lost and that errors are handled systematically, improving reliability of the pipeline.

35. How can Composer coordinate a Pub/Sub → Dataflow → BigQuery pipeline?

Composer acts as the orchestrator of the whole pipeline. I would design the DAG with tasks like:

1. Trigger a Dataflow job that reads messages from Pub/Sub.
2. Wait for the Dataflow job to complete or monitor its status.
3. Once data is processed, run validation queries in BigQuery to check row counts, schema consistency, or data quality.
4. If validation passes, move raw files from staging to archive in GCS.
5. If validation fails, trigger alerting and stop further processing.

Composer ensures each step happens in sequence and with the right dependencies. It also manages retries, monitoring, and error handling, making the Pub/Sub → Dataflow → BigQuery pipeline production-ready

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

36. When should ordering keys be used in Pub/Sub?

Ordering keys should be used when the sequence of messages really matters for downstream processing. By default, Pub/Sub delivers messages in parallel without guaranteeing order. But if I have a use case like financial transactions, inventory updates, or IoT device signals where the order of events must be preserved, I add an ordering key.

For example, all events for customer_id=123 can be published with the same ordering key. Pub/Sub then ensures that those events are delivered in the exact order they were published. This is especially important in systems where out-of-order processing can cause wrong balances, incorrect states, or data corruption.

37. What trade-offs occur when enabling ordered message delivery?

The main trade-off is throughput and parallelism. When I enable ordering keys, Pub/Sub has to process messages with the same key in strict sequence. This reduces the level of parallelism because Pub/Sub cannot deliver multiple messages for the same key at the same time.

If a subscriber crashes or fails to ack one message, all later messages with the same key are held back until that one is successfully delivered. This increases latency for that key.

So ordering gives correctness but at the cost of reduced throughput and potentially higher latency. That's why I only enable ordering where it's strictly necessary, not for all data.

38. How can duplicates be handled in streaming pipelines consuming Pub/Sub?

Since Pub/Sub guarantees at-least-once delivery, duplicates can happen. To handle them, pipelines must be designed with deduplication or idempotency.

A common approach is to use a unique message ID (either provided by Pub/Sub or embedded in the payload). Downstream systems like BigQuery or databases can use this ID to ignore duplicates.

In Dataflow, I can also apply deduplication transforms that keep track of recently seen IDs within a time window. Another strategy is to write raw data into GCS or BigQuery staging tables and then run a deduplication query before promoting it to the curated dataset.

The key point is that duplicates are expected, so the pipeline must be resilient to them.

39. Why should subscribers be idempotent when consuming Pub/Sub data?

Idempotency means that even if the same message is processed multiple times, the end result is the same. This is critical because Pub/Sub may deliver a message more than once due to retries, missed acknowledgments, or network issues.

For example, if a message says "add $100 to account," running it twice would cause an error. But if the message says "set account balance to $100" or includes a unique transaction ID that ensures it only applies once, then duplicates won't cause harm.

Designing idempotent subscribers ensures that the pipeline produces consistent results even in the face of retries or duplicates, which is essential for reliability.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

40. How does Dataflow provide exactly-once semantics when writing Pub/Sub data to BigQuery?

Pub/Sub itself provides at-least-once delivery, but Dataflow can add exactly-once guarantees for sinks like BigQuery by combining three features:

1. Checkpointing: Dataflow keeps track of which messages have already been processed, so if a worker fails, it restarts from the last checkpoint without reprocessing the same batch.
2. Deduplication: Dataflow uses unique Pub/Sub message IDs to avoid writing the same message twice when retries happen.
3. BigQuery streaming with Storage Write API: When using the Storage Write API, Dataflow can ensure exactly-once writes because BigQuery itself deduplicates based on insert IDs.

This combination means that even though Pub/Sub may send duplicates, the Dataflow-to-BigQuery pipeline will only write each message once to the final table.

41. What is the purpose of a dead-letter queue in Pub/Sub?

The purpose of a dead-letter queue (DLQ) is to make sure that problematic messages don't get lost or block the pipeline. Sometimes a message cannot be processed by a subscriber even after multiple retries, maybe because it's malformed, has missing data, or keeps causing errors in downstream systems.

Instead of retrying endlessly or dropping the message silently, Pub/Sub moves it to a separate DLQ topic. This allows me to inspect those messages later, fix them, and reprocess them if needed. DLQs improve reliability, traceability, and make pipelines more resilient.

42. How can failed events be routed into backup topics?

In Pub/Sub, I can configure a subscription with a dead-letter policy. This policy says that after a message has been retried a certain number of times without being acknowledged, it should be moved to a specified backup topic (the DLQ).

From there, another subscription can be attached to the DLQ for monitoring, alerts, or storage in GCS/BigQuery. This way, I don't lose failed events, and I can analyze them separately to understand why they failed.

So essentially, failed events get automatically rerouted into a backup topic based on retry limits I set.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

43. How can DLQ messages be reprocessed in Pub/Sub?

Reprocessing DLQ messages depends on how the pipeline is designed. The typical approach is:

1. Create a subscription to the DLQ topic.
2. Set up a separate processing pipeline (maybe another Dataflow job or Cloud Function) that consumes from the DLQ.
3. Clean or transform the messages to fix the issue (like correcting schema mismatches or filling missing fields).
4. Republish them into the original Pub/Sub topic or directly load them into BigQuery or GCS.

This way, instead of dropping bad messages, I give them a second chance to be processed after fixing the underlying issue.

44. How do you differentiate between transient and permanent errors in message handling?

Transient errors are temporary issues that usually succeed if retried. Examples include network timeouts, database connection issues, or temporary API throttling. These can be solved with retries and backoff.

Permanent errors are caused by issues that won't go away with retries, like invalid schema, corrupted data, or missing mandatory fields. These need manual intervention or transformation.

In Pub/Sub, I design subscribers to retry on transient errors within the ack deadline. If errors keep happening beyond the retry limit, the message is treated as a permanent error and moved to a DLQ. This separation ensures that only truly unprocessable messages end up in DLQ.

45. How can DLQs help monitor data quality issues in a pipeline?

DLQs are not just for error handling, they also act as a signal for data quality. If too many messages are routed to DLQ, it tells me that something is wrong upstream — maybe schema drift, unexpected nulls, or incorrect payload structure.

By analyzing DLQ messages, I can identify patterns in bad data and update validation logic, schemas, or source systems to prevent the same issues in the future. I can also set alerts in Cloud Monitoring so the team knows immediately if DLQ volumes spike.

This turns DLQs into a powerful tool for monitoring and improving data quality over time.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

46. How does Pub/Sub handle late-arriving messages?

Pub/Sub itself doesn't really have a concept of "late." It just guarantees that all published messages will be delivered at least once to subscribers, as long as they are within the retention window (default 7 days, extendable to 31).

It's actually the downstream consumer, like Dataflow, that decides whether a message is late based on event-time stamps. Pub/Sub will deliver the message no matter when it arrives, but Dataflow may decide it belongs to an earlier window that already closed. That's where watermarking and allowed lateness come into play.

So, in simple terms: Pub/Sub delivers messages reliably; late handling is the responsibility of the processing engine.

47. What role do watermarks play when consuming Pub/Sub messages in Dataflow?

Watermarks are Dataflow's way of tracking event-time progress. Since Pub/Sub messages can arrive out of order or with delays, Dataflow needs a way to know when it's safe to close a window and output results.

The watermark is an estimate of the "minimum timestamp of future events." For example, if the watermark for a stream is 12:00, Dataflow assumes that most messages earlier than 12:00 have already been seen, and it can safely close windows up to that point.

Without watermarks, Dataflow wouldn't know when to emit results. Watermarks balance correctness and timeliness by deciding when to finalize windowed computations.

48. How can allowed lateness be configured in Dataflow with Pub/Sub messages?

Allowed lateness is a setting in Dataflow that specifies how long to wait after the watermark before discarding late messages. For example, if I set allowed lateness to 10 minutes, then even after a window closes, Dataflow will still accept and update results with messages that arrive up to 10 minutes late.

This is important when using Pub/Sub because events often arrive delayed due to network, retries, or batching. If I don't configure allowed lateness, those messages would be dropped.

Configuring allowed lateness ensures I balance result accuracy with system performance. Longer lateness improves correctness but means windows remain open longer, increasing memory and processing overhead.

49. How do out-of-order events impact streaming aggregations?

Out-of-order events can cause inaccurate results if not handled properly. For example, if I'm calculating the number of clicks per minute and a late event arrives for a minute that already closed, the count will be wrong if I don't account for it.

This is why windowing, watermarks, and allowed lateness are important. They allow the pipeline to adjust aggregates when late data comes in. However, if lateness is too high, it can delay results or require expensive recomputation.

So, out-of-order events make streaming pipelines more complex — I need to carefully design windowing and lateness handling to ensure correctness while still providing timely results.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

50. How can resilient pipelines be designed for event-time correctness with Pub/Sub?

To design resilient pipelines that process data correctly based on event time, I usually combine several strategies:

1. Include event-time timestamps in every Pub/Sub message (not just processing time).
2. Use Dataflow's event-time windowing with watermarks so results are aligned to when events actually happened, not when they were processed.
3. Configure allowed lateness to handle delayed or retried messages.
4. Make aggregations and transformations idempotent so that reprocessing or updates don't cause duplication.
5. Keep raw Pub/Sub data archived in GCS so I can reprocess if needed (for example, if lateness was larger than expected).

By combining these, the pipeline produces results that are both timely and accurate, even when events arrive late or out of order.

51. How does Pub/Sub scale horizontally for subscribers?

Pub/Sub scales horizontally by allowing multiple subscriber clients to pull from the same subscription in parallel. The messages are distributed fairly across subscribers, and Pub/Sub automatically balances the load.

For example, if I have a subscription with 10 subscribers, each subscriber will get a portion of the messages. This means I can scale subscribers horizontally just by adding more worker instances, and Pub/Sub will handle the distribution.

On top of that, Pub/Sub partitions the topic internally across many servers, so the system can handle millions of messages per second and deliver them efficiently to subscribers worldwide.

52. How can message flow be controlled using ack deadlines?

The ack deadline is the time a subscriber has to acknowledge a message. By default, it's 10 seconds, but it can be extended up to 10 minutes.

If processing takes longer, the subscriber can extend the deadline to prevent Pub/Sub from redelivering the message. If the subscriber doesn't ack within the deadline, Pub/Sub assumes it failed and will resend it.

This lets me control message flow. For short tasks, I keep the default. For long-running processing, I extend deadlines dynamically. If I want faster retries for unprocessed messages, I shorten the ack deadline. This way, I balance reliability and throughput based on processing needs.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

53. What happens if there is a large backlog of Pub/Sub messages?

When there's a backlog, it means subscribers are not keeping up with publishers. Pub/Sub will keep storing undelivered messages up to the retention limit (7 to 31 days). As the backlog grows, subscribers may see increased latency because they are pulling older messages while new ones keep arriving.

To clear a backlog, I can scale out subscribers (add more workers or increase Dataflow parallelism). Pub/Sub itself is built to handle very large backlogs, but downstream systems might struggle if they can't process fast enough.

If a backlog persists beyond retention, older messages will expire and be lost, so it's important to monitor backlog metrics and scale subscribers proactively.

54. How can parallelism be tuned in Dataflow pipelines consuming Pub/Sub?

Dataflow automatically scales workers based on load, but I can tune parallelism by configuring settings like the number of workers, max workers, and autoscaling policies.

For Pub/Sub specifically, Dataflow creates multiple readers per subscription. Each reader pulls messages in parallel, and more workers mean more readers. I can also use multiple subscriptions (fan-out pattern) to spread the load further.

To optimize parallelism, I balance the number of workers against the processing complexity. For lightweight transformations, I can increase workers aggressively. For heavy processing, I scale more gradually to avoid overwhelming downstream sinks like BigQuery or GCS.

55. How do partitioned topics help increase throughput in Pub/Sub?

A partitioned topic spreads messages across multiple partitions inside Pub/Sub. Each partition can handle a high message rate, and multiple partitions combined give very high throughput.

Subscribers then read from all partitions in parallel, which increases overall throughput. Partitioned topics are especially useful when I have very high-volume publishers and need to guarantee ordering per key while still scaling horizontally.

For example, if I use an ordering key like customer_id, Pub/Sub ensures ordering within each partition for that key, but still distributes different keys across partitions. This gives both scalability and ordered delivery.

56. What is the default message retention period in Pub/Sub?

By default, Pub/Sub retains undelivered messages for 7 days. This means if a subscriber is offline or lagging, it has up to 7 days to catch up and still receive the messages. If the message is acknowledged, it's removed immediately from the subscription.

So in normal operation, messages don't sit around long, but if there's a failure or downtime, the 7-day retention gives a buffer to recover without losing data.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

57. When would you extend message retention to 7–31 days?

I extend retention when subscribers may be offline for a long time or when I need a safety net for reprocessing.

For example, in compliance-heavy industries like finance or healthcare, I might extend retention to ensure no data is lost even if downstream systems are down for days. It's also useful when pipelines need to be paused for maintenance or upgrades — longer retention ensures I don't lose events while systems are stopped.

Another case is debugging or replaying historical data. With longer retention, I can reattach a subscription and replay events up to 31 days old without having to rebuild from raw storage.

58. How can messages be replayed in Pub/Sub for reprocessing?

Pub/Sub doesn't store messages forever, but within the retention window, I can replay them. The way to do this is to create a new subscription or seek an existing subscription back to an earlier timestamp or snapshot.

For example, if I want to reprocess all events from yesterday, I can use the seek feature to reset the subscription pointer back to that time. Then the subscriber will read all those messages again.

This is very useful for recovery after pipeline failures, schema changes, or reloading data into BigQuery with new transformations.

59. What are the cost trade-offs of extending message retention in Pub/Sub?

Extending retention increases storage costs. Pub/Sub charges for storing undelivered messages, so keeping them for 31 days instead of 7 means higher costs, especially for high-volume topics.

There's also an indirect cost: large backlogs mean subscribers need more time and resources to catch up, which increases processing costs in Dataflow or other consumers.

So retention is a balance — longer retention improves resilience and flexibility, but it's more expensive. That's why I usually pair extended retention with archiving to GCS, which is cheaper for long-term storage.

60. How is retention different from using DLQs in Pub/Sub pipelines?

Retention is about how long undelivered messages are kept in the system. It ensures subscribers have time to catch up or replay.

A DLQ (Dead Letter Queue) is about handling failed messages that subscribers couldn't process successfully, even after retries. Instead of retrying forever, those bad messages are moved to a DLQ topic for analysis and reprocessing.

So retention deals with timing and backlog, while DLQ deals with unprocessable messages. In practice, I use both: retention to give resilience against downtime, and DLQs to capture data quality issues without losing messages.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

61. How can schema enforcement be applied to Pub/Sub messages?

Pub/Sub supports schema enforcement using its built-in schema registry. I can define a schema in formats like Avro or Protocol Buffers, then attach that schema to a topic. Once a schema is attached, Pub/Sub will validate every published message against that schema.

If the message doesn't match, Pub/Sub rejects it right at publish time. This prevents bad or malformed messages from entering the pipeline. Consumers then have confidence that all messages they read are valid and consistent.

This enforcement makes downstream processing easier because I don't need to handle unexpected message structures at the consumer level.

62. What happens if a malformed JSON message is published into Pub/Sub?

If there is no schema attached to the topic, Pub/Sub will accept the malformed JSON because it treats the payload as raw bytes. Consumers may then fail when trying to parse it.

If a schema is enforced (like Avro or Protobuf), then the malformed JSON will be rejected immediately at publish time with an error. This means the producer gets feedback and knows the message didn't go through.

So whether malformed messages get through depends on whether schema enforcement is enabled on the topic. Without it, the burden shifts to consumers to detect and handle bad payloads.

63. How do producers and consumers handle schema evolution in Pub/Sub?

Schema evolution means fields are added, removed, or changed over time. In Pub/Sub, the schema registry supports backward and forward compatibility rules.

Producers handle schema evolution by publishing with the new schema version, making sure it's compatible with older versions (for example, only adding optional fields).

Consumers can choose which version of the schema to use. If they're reading older messages, they can still process them as long as the changes are compatible. For example, if a new field is added, older messages will just have null for that field.

The key is coordination: producers should evolve schemas carefully, and consumers should be able to handle both old and new versions gracefully.

64. What is the Pub/Sub schema registry, and how is it used?

The schema registry is a feature in Pub/Sub that stores and manages message schemas in a centralized way. I can define a schema in Avro or Protobuf, register it in the schema registry, and then attach it to one or more topics.

When publishing, producers must send data that conforms to the schema. When consuming, subscribers can fetch the schema from the registry to deserialize the message correctly.

This ensures consistency across producers and consumers, makes it easier to evolve schemas, and prevents schema drift. It also improves reliability, since invalid messages are caught at the publishing stage.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

65. Why is logging invalid messages important in streaming pipelines?

Logging invalid messages is crucial because it gives visibility into what's going wrong. If a message is malformed, has the wrong schema, or contains bad data, I don't want it to disappear silently.

By logging invalid messages into a separate GCS bucket or BigQuery table, I can analyze patterns — for example, maybe a new producer version started sending bad data. Logging also helps with debugging, compliance, and replaying messages once they are fixed.

In production, I usually send invalid messages to a dead-letter queue or a "rejects" folder in GCS, with logs attached. This way, the pipeline stays healthy while bad data is captured for review and reprocessing.

66. How would you design a fraud detection pipeline using Pub/Sub?

Fraud detection requires very low latency because suspicious transactions must be flagged immediately. I would design it like this:

1. Applications (like payment systems) publish each transaction event into a Pub/Sub topic.
2. A Dataflow streaming pipeline consumes these messages in real time, enriches them with customer history or geolocation reference data, and applies rules or machine learning models to detect anomalies.
3. If a transaction is flagged as suspicious, Dataflow can publish an alert into another Pub/Sub topic that triggers a Cloud Function or Cloud Run service for instant action, like blocking the transaction.
4. At the same time, all raw events are archived into GCS and written into BigQuery for long-term analysis and model retraining.

This design uses Pub/Sub as the backbone for event distribution, making fraud detection scalable, fast, and reliable.

67. How can Pub/Sub be used for clickstream analytics?

Clickstream data comes from websites and mobile apps, often generating millions of events per second. I would publish each click or interaction into a Pub/Sub topic.

From there, a Dataflow pipeline can consume the events, apply windowing (for example, 1-minute windows) to aggregate counts like page views, sessions, or clicks per product, and then stream results into BigQuery for near real-time analytics.

The raw clickstream can also be stored in GCS for deeper batch processing later using Dataproc or BigQuery ML. Pub/Sub ensures that no matter how spiky the traffic is, all events are captured and distributed reliably to multiple consumers, like analytics dashboards, personalization systems, and ML pipelines.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

68. What role does Pub/Sub play in IoT monitoring pipelines?

In IoT systems, devices like sensors, meters, or trackers constantly produce telemetry data. Pub/Sub is the ingestion layer that collects all this data in real time.

Each device or gateway publishes messages to a Pub/Sub topic. Pub/Sub then buffers and distributes those messages to multiple consumers:

- A Dataflow pipeline that processes and stores metrics in BigQuery.
- A monitoring system that triggers alerts if readings go outside safe thresholds.
- A GCS sink where all raw IoT messages are archived for long-term analysis.

Pub/Sub is critical here because it can handle millions of concurrent device messages, smooth out traffic spikes, and fan out data to multiple downstream systems simultaneously.

69. How can Pub/Sub power real-time dashboards with BigQuery and Looker Studio?

The flow looks like this:

1. Pub/Sub receives streaming events like sales transactions, user activity, or IoT readings.
2. A Dataflow pipeline consumes those messages and writes them directly into BigQuery using the Storage Write API.
3. Since BigQuery tables update in near real time, Looker Studio dashboards connected to BigQuery can reflect the latest data within seconds.

This setup is powerful for operational dashboards — for example, monitoring live orders in an e-commerce site or live sensor readings in a factory. Pub/Sub ensures real-time ingestion, BigQuery handles scalable storage and queries, and Looker Studio gives visualization.

70. Why are hybrid batch + streaming pipelines important for business analytics?

Not all data needs to be real-time. Some use cases, like fraud detection or live dashboards, demand streaming. But others, like monthly revenue reporting or model retraining, can be handled in batch.

Hybrid pipelines let me combine both. For example, Pub/Sub + Dataflow provides fresh data continuously for dashboards and alerts, while GCS + BigQuery batch jobs provide reliable historical datasets for trend analysis.

This balance is important for cost and practicality. Running everything in streaming mode would be expensive and over-engineered, while relying only on batch would make the business blind to real-time opportunities. Hybrid pipelines ensure businesses get both speed and depth in their analytics.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

71. What are the key steps in migrating Kafka workloads into Pub/Sub?

First, treat migration as a project with discovery, compatibility checks, pilot, and cutover phases — not a single command. I usually follow these steps:

1. Inventory and assess

- Catalog topics, partitions, producers, consumers, retention settings, message formats, throughput patterns, and SLAs.
- Identify features used in Kafka that matter (ordering per key, exactly-once semantics, transactions, log compaction, consumer lag behavior).

2. Map capabilities

- Match Kafka concepts to Pub/Sub equivalents: topics → topics, partitions → (Pub/Sub partitions or ordering keys), consumers → subscriptions, retention → message retention (7–31 days).
- Note gaps: Kafka consumer groups and offsets differ from Pub/Sub ack semantics; Kafka transactions aren't natively supported in Pub/Sub.

3. Design the target architecture

- Decide whether to keep the same logical topic names, how to handle ordering (ordering keys or partitioned topics), and how to handle consumer groups (multiple subscriptions).
- Choose integration patterns: direct migration (connectors) or hybrid (mirror, dual-write).

4. Choose migration approach

- For low-downtime migration: use a mirroring pattern (tools or Dataflow connectors) to replicate Kafka topics into Pub/Sub in near real time.
- For controlled migration: use dual writes (producers write to both Kafka and Pub/Sub) and switch consumers gradually to Pub/Sub.
- For bulk historical data: export Kafka logs and load them into GCS, then publish into Pub/Sub or process via batch jobs.

5. Implement bridging and compatibility

- Use connectors: Dataflow has KafkaIO for reading from Kafka and Pub/SubIO for writing to Pub/Sub; Confluent tools or MirrorMaker alternatives can help.
- Normalize message formats (prefer Avro/Protobuf) and use schema registry strategies (Avro/Schema Registry → Pub/Sub Schema) to avoid format drift.

6. Pilot and validate

- Start with non-critical topics, validate end-to-end latency, ordering, throughput, retry behavior, and consumer processing.
- Verify monitoring, metrics, and alerting work similarly (consumer lag → subscription backlog, etc.).

7. Cutover

- Gradually switch consumers: move one consumer group at a time from Kafka to Pub/Sub, validate, and decommission Kafka consumers.
- If dual-write was used, stop writing to Kafka only after all consumers have moved and you're confident in Pub/Sub behavior.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

8. Post-migration hardening

- Tune Pub/Sub retention, partitioning, and autoscaling; set up DLQs, monitoring, and cost controls.
- Optimize message formats, use Storage Write API or Dataflow where appropriate for BigQuery sinks.

Example snippet for a practical migration: set up a Dataflow job that reads from Kafka and writes to Pub/Sub, run in parallel with Kafka production for a week to validate, then switch consumers.

The key is incremental migration with replayability: archive original Kafka data so you can reprocess if something unexpected arises.

72. How can hybrid pipelines use both Kafka and Pub/Sub?

Hybrid pipelines are common during migrations or in multi-cloud/edge architectures. The main patterns I use:

1. Kafka at the edge / on-prem + Pub/Sub in cloud (ingest bridge)

- Keep low-latency local Kafka for on-prem producers and consumers. Use a bridge (Dataflow Kafka → Pub/Sub connector or MirrorMaker-like tool) to replicate events to Pub/Sub for cloud consumers, analytics, and ML.
- This preserves local performance while enabling cloud-native processing.

2. Fan-out and domain separation

- Use Kafka for internal microservices communication where sticky ordering and compacted logs are important, and Pub/Sub for external/analytics fan-out to many cloud services (BigQuery, Dataflow, Cloud Functions).
- Publishers write to Kafka; a replication job streams a curated subset into Pub/Sub for analytics pipelines.

3. Dual-write for gradual migration or multi-sink publication

- Producers write events to both Kafka and Pub/Sub. Kafka consumers keep reading as before, while new cloud consumers read from Pub/Sub. Over time, move consumers off Kafka.
- Useful where producers cannot easily be changed to write only to Pub/Sub initially.

4. Federated processing and transformation

- Use Kafka for OLTP/event sourcing; a transformer job reads Kafka, enriches events, and publishes normalized messages to Pub/Sub for data lake and BI workflows.

Operational considerations

- Ensure consistent message schema and IDs across both systems to allow de-duplication and reconciliation.
- Manage ordering: if ordering is required, design keys and partitioning so both systems can preserve the same ordering guarantees (Pub/Sub ordering keys + partitioned topics vs Kafka partitions).
- Monitor end-to-end latency and duplication; use unique IDs and idempotent consumers to handle replay/duplicates.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Concrete example: an IoT gateway cluster uses Kafka for local ingestion and immediate control loops; a Dataflow job mirrors data to Pub/Sub for long-term analytics and alerts (BigQuery dashboards), allowing cloud teams to build without touching on-prem Kafka.

73. What migration strategies exist for moving from Kafka to Pub/Sub (mirror topics, dual writes)?

There are three practical strategies I recommend, chosen by risk appetite and downtime tolerance:

1. Mirror (replication)

- Set up a continuous replication job that reads Kafka topics and writes to Pub/Sub topics in near real time.
- Tools: Dataflow KafkaIO → Pub/SubIO, Confluent Replicator (customized), or open-source MirrorMaker with adapters.
- Pros: Low consumer disruption (consumers can switch at leisure), single source of truth preserved in Kafka during transition.
- Cons: Operational overhead for running replication, careful handling of offsets and ordering needed.

2. Dual writes (parallel publishing)

- Update producers to publish to both Kafka and Pub/Sub simultaneously.
- Gradually move consumers to Pub/Sub and validate.
- Pros: Controlled migration, easy rollback (consumers still get Kafka messages).
- Cons: Increased producer complexity, risk of temporary divergence if one write fails.

3. Bulk export + replay

- Export historical Kafka logs (e.g., via Kafka export tools or consuming from the earliest offset) and push them into GCS or directly into Pub/Sub for backfill.
- Then switch to live replication/dual-write for real-time sync.
- Pros: Good for initial backfills and bootstrapping historical datasets in GCS/BigQuery.
- Cons: Doesn't solve live cutover by itself; needs combination with mirror or dual writes.

Choosing strategy

- For minimal risk and gradual change: do mirror + backfill, then move consumers.
- For fast migration with coordinated release windows: dual-write and coordinated consumer switch.
- For one-time migrations or archival: export and bulk load.

Important during all strategies

- Maintain unique message IDs for deduplication.
- Normalize schema and register it in Pub/Sub schema registry or an equivalent.
- Validate throughput and latency of the replication path.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

74. How can legacy batch ETL pipelines be modernized into event-driven Pub/Sub pipelines?

Modernization is about shifting from periodic large jobs to near-real-time, decoupled processing while preserving reliability and correctness. My approach:

1. Understand the existing batch flows

- Identify data sources, frequencies, dependencies, schema, SLAs, and why batch was chosen (e.g., cost, source constraints).

2. Decompose into events

- Where possible, change sources to emit events (CDC for databases, application events for changes, file notifications for new dumps).
- Use change-data-capture (CDC) tools (e.g., Debezium, Datastream) to stream DB changes into Kafka or Pub/Sub rather than doing nightly full extracts.

3. Introduce Pub/Sub as the ingestion layer

- Have producers (apps, CDC tools) publish change events or domain events to Pub/Sub topics. Standardize event schemas (Avro/Protobuf) and include metadata (timestamps, IDs, operation type).

4. Move transformations to streaming or micro-batch processors

- Use Dataflow (Beam) for streaming ETL where you can validate, enrich, window, and write to sinks like BigQuery or GCS.
- For heavy historical transformations, use a hybrid approach: stream small changes for freshness, and run scheduled batch jobs for bulk aggregation.

5. Ensure idempotency and ordering

- Design consumers to be idempotent and include dedupe logic using unique IDs. Use ordering keys only where the sequence matters.

6. Provide fallbacks and reprocessing

- Keep raw events archived in GCS as a replay source. Support seek/replay for Pub/Sub to reprocess windows.
- Implement DLQs and monitoring to capture and repair bad events.

7. Refactor orchestration

- Replace monolithic schedulers with event-driven orchestration (Composer + triggers, or direct Pub/Sub → Cloud Functions → Dataflow chains).

8. Gradual cutover

- Start with low-risk flows, implement dual-write to both batch and streaming, validate outputs, then move to full streaming.

Business benefits

- Lower time-to-insight, more resilient decoupling between teams, incremental processing cost optimizations, and better support for real-time use cases (alerts, personalization).

Example: replace daily ETL that loaded DB snapshots into BigQuery with CDC feeding Pub/Sub → Dataflow to apply transforms and stream updates into BigQuery via the Storage Write API, while keeping a nightly full-snapshot job for reconciliation for a month during cutover.

75. How does Pub/Sub support enterprise event-driven architecture modernization?

Pub/Sub is well-suited for enterprise modernization because it provides a scalable, managed messaging backbone that enables loose coupling, fan-out, and integration across cloud services. Key ways it supports modernization:

1. Decoupling and scalability

- Producers publish events without knowing consumers. This allows teams to evolve independently and scale consumers horizontally without changing producers.

2. Fan-out and multimodal consumption

- A single event can be consumed by multiple systems simultaneously (analytics, ML, monitoring, OLTP). Pub/Sub supports many subscribers per topic, enabling event-driven microservices and analytics to run in parallel.

3. Managed durability and autoscaling

- Pub/Sub handles storage, partitioning, and autoscaling so teams focus on business logic rather than provisioning brokers. It also offers retention windows and DLQs for resilience.

4. Integration with cloud services

- Native connectors to Dataflow, Cloud Functions, Cloud Run, BigQuery (Storage Write API), and sinks like GCS make building pipelines fast and consistent across the organization.

5. Supports hybrid and migration scenarios

- Pub/Sub can integrate with on-prem Kafka via connectors or Dataflow, enabling gradual modernization without ripping out legacy systems immediately.

6. Schema registry and governance

- Topic-level schemas and a registry help enforce contract-first designs and prevent schema drift across producers and consumers, which is essential at enterprise scale.

7. Operational observability

- Built-in metrics (ack rates, backlog) and integration with Cloud Monitoring allow enterprise teams to monitor SLAs and platform health centrally.

8. Security and compliance

- IAM-based access control, CMEK support, and VPC-SC integrations allow enterprises to meet strict security and compliance requirements.

Business outcome

- Pub/Sub helps enterprises move from monolithic, batch-driven architectures to responsive, event-driven systems where teams can deliver features faster, scale independently, and enable near-real-time analytics and automation across the company.

Example elevator pitch: move from nightly monoliths to event-driven microservices by publishing domain events to Pub/Sub, processing them via Dataflow for analytics and Cloud Run for business workflows — enabling real-time dashboards, faster reaction to anomalies, and simpler team boundaries.

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar